Decisionhouse: Prescriptive Analytics in the Data Stack

Matteo Brucato Fjodor Kholodkov Soren Little Jakob Mayer Duc Nguyen
OSM Data
{matteo, fjodor, soren, jakob, duc}@osm-data.com

ABSTRACT

Data platforms have evolved by making data-intensive workloads native: SQL and query optimizers eliminated bespoke data-retrieval programs; Lakehouses added first-class support for ML training and serving over the same data. Prescriptive analytics—computing optimal actions subject to constraints over data—is equally data-intensive, yet remains outside the platform: every optimization problem requires a hand-built pipeline from data extraction to solver invocation, rebuilt from scratch whenever the data or the requirements change. We propose *Decisionhouses*, a new class of data infrastructure that makes prescriptive analytics *native*. A Decisionhouse provides (i) DeQL (Decision Query Language), a declarative SQL extension where users express decision problems over relational data; (ii) automatic formulation selection that exploits query and data semantics to pick the right problem class and solver—a choice that can change a query's complexity class from NP-hard to polynomial; and (iii) end-to-end integration of optimization into the data platform, from query parsing through solver execution. Decisionhouses can help address several challenges that have kept optimization outside data platforms, including pipeline brittleness, formulation expertise, structural blindness, and scalability cliffs, and make decision-making as accessible as querying data.

PVLDB Reference Format:
Matteo Brucato Fjodor Kholodkov Soren Little Jakob Mayer Duc Nguyen. Decisionhouse. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

1 INTRODUCTION

A database can answer "what does it cost to ship from each warehouse to each store?"—a join. It cannot answer "which routes should we activate to minimize total cost while meeting every store's demand without exceeding any warehouse's supply?" The data is the same tables. Yet answering requires leaving the database to build a bespoke pipeline that transforms data, encodes a mathematical model, runs a solver, and maps results back.

These are *prescriptive* questions: given options from relational data, constraints from business policy, and a measurable objective, compute the best course of action [50]. Descriptive analytics has declarative infrastructure: diverse questions share one system (the data warehouse [9, 17, 42]) and one language (SQL). Predictive analytics followed suit with ML platforms and feature stores [19, 64]. Prescriptive analytics has no such common infrastructure.

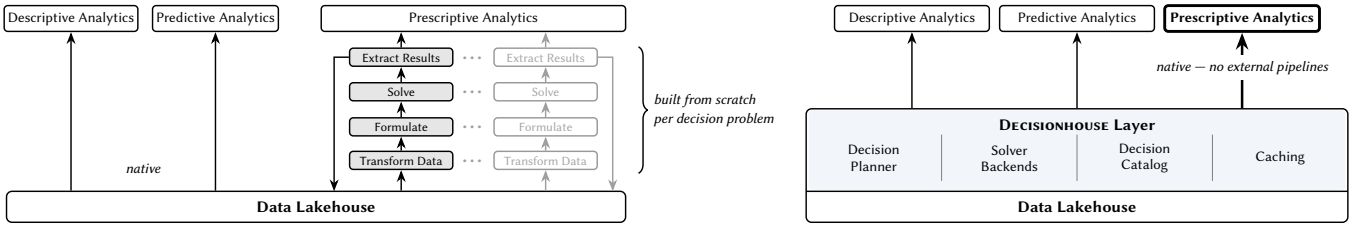
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

The reason is structural: each optimization problem maps to a different mathematical formulation—shipping requires a network flow model, scheduling requires an assignment model—and crossing from relational data to the right formulation is the *data-to-formulation gap*. Once formulated, invoking the solver and mapping results back are mechanical; the bottleneck is this translation, which changes for every problem and requires optimization expertise. N problems create N independent pipelines (Figure 1a).

Descriptive queries all operate over a single relational abstraction; ML problems share a uniform shape (features → model → prediction) that MLOps platforms standardize [19]. Optimization has no such uniformity. Modeling languages [22, 28] and OR platforms [12, 21] are declarative over model parameters, but the parameters must still be populated manually from data—they bridge the formulation-to-solver gap, not the data-to-formulation gap. PaQL [14, 27, 40] closed the gap for one problem class (package queries) but does not generalize. LLM-based tools translate natural language into optimization scripts, but each script still embeds its own data pipeline; the LLM automates writing the pipeline, not eliminating it. We call the accumulated cost *decision debt*: without shared infrastructure, each team independently reinvents extraction, formulation, and solver integration.

We define *Decisionhouses* as a class of data infrastructure for prescriptive analytics that internalizes per-problem pipelines behind a declarative language (Figure 1b, Section 2). We propose *DeQL* (Decision Query Language) as the declarative interface (Section 3). DeQL extends SQL with optimization semantics—decision variables, constraints with grouping, and objectives over aggregate expressions—so that analysts express decision problems directly over relational data. Each optimization problem requires a different DeQL query, not a different pipeline—the same distinction as writing different `SELECT`s versus writing different data programs. Because every decision query produces a standard relation, results compose with SQL and with other decision queries without a separate result-mapping step. Internally, the Decisionhouse layer provides a decision planner that selects formulations and solvers automatically, pluggable solver backends, a decision catalog for governance and audit trails, and incremental caching for interactive what-if analysis (Figure 1b).

SQL queries process tuples independently; optimization requires global coordination across all candidates through shared constraints—each decision depends on all others, so even polynomial problems like network flow need the full candidate space. Generic mixed-integer programming adds NP-hard combinatorial search on top of this coordination. Recognizing problem structure can change the effective *complexity class* (Section 4.2): in the shipping example, the grouped constraints (supply per warehouse, demand per store) and the bipartite key structure (warehouse, store) identify a transportation problem solvable as minimum-cost flow [5] in polynomial time. The challenge is that solvers receive only numeric matrices—grouped constraints become anonymous rows, bipartite



(a) Today (b) With a Decisionhouse

Figure 1: Prescriptive analytics in data platforms: per-problem external pipelines today (a) vs. native in a Decisionhouse (b).

keys become anonymous columns, and the structural patterns that identify network flow are lost. Solvers attempt limited structure detection on these matrices [26, 34], but a declarative query retains the original entity types, grouping patterns, and database statistics that solvers never see. For NP-hard instances, abstraction-based decomposition [14, 40, 45] partitions the candidate space into coarser groups, reducing solver input by orders of magnitude while bounding approximation error.

Structure-aware formulation selection can determine whether a decision query succeeds at all. On a GPU allocation query—a supply-demand problem with the same structure as the shipping example—selecting min-cost flow over generic MILP (solved by HiGHS [31]) yields 29–355× speedup at up to 1.8M candidate assignments; at 7.4M, MILP times out after 10 minutes while min-cost flow solves in 2.2 seconds (Section 4.3). All three formulations produce identical optimal solutions—total unimodularity of the transportation matrix guarantees integer optima across formulations [5]—so the speedup comes entirely from structure recognition, not approximation.

This paper defines **eight requirements** for prescriptive data infrastructure by comparing existing systems and identifying the gaps each leaves open (Section 2), presents **DeQL** (Section 3) and a **Decisionhouse architecture** with a three-IR compilation pipeline and automatic formulation selection (Section 4), and outlines open problems at the intersection of database systems, combinatorial optimization, and scalable solving (Section 5). This work extends prior research on in-database prescriptive analytics [14, 15, 27, 40] from single-class package queries to a general-purpose system, shaped by our ongoing industry collaborations in logistics, process mining, and cloud platforms.

2 THE DECISIONHOUSE

The data-to-formulation gap (Section 1) explains why prescriptive analytics lacks common infrastructure. Three developments make closing this gap feasible and timely. (a) *Accessible data.* The lakehouse architecture [64] and open table formats (Parquet, Delta Lake [8]) decoupled organizational data from the database engine that hosts it, so external systems can read it directly. (b) *Existence proof.* PaQL [14] closed the data-to-formulation gap for package queries; abstraction-based decomposition [14, 27, 40] scales it to millions of candidates, with extensions to probabilistic constraints [15, 41]. The Decisionhouse generalizes across problem types. (c) *Generated code needs runtime.* LLM-based tools [4, 29, 38, 65] generate per-problem optimization scripts; a shared execution layer would provide data integration, formulation selection, and governance once, rather than per script.

Table 1: No existing system addresses all Decisionhouse requirements. ✓ addressed, ○ partial, ✗ not in scope. [1]Requires OR expertise. [2]Data connectors, but not integrated in the data layer. [3]Generic interface, not extensible. [4]Binary selection only. [5]Logging available, not decision-specific. [6]LLM-generated formulation may be incorrect. [7]Solver-level time limits only. [8]Scalable, no progress feedback.

Requirement	Solvers	Modeling langs	OR Platf.	PaQL	SolveDB	LLM	Decision-house
(R1) Accessible	✗	○[1]	○[1]	✓	○[1]	✓	✓
(R2) Data-native	✗	✗	○[2]	✓	✓	✗	✓
(R3) Extensible	✓	✓	✓	✗	○[3]	✓	✓
(R4) Composable	✗	✗	✗	○[4]	✓	✗	✓
(R5) Auto-plan	✗	✗	✗	✗	✗	✗	✓
(R6) Governance	✗	✗	○[5]	○[5]	○[5]	✗	✓
(R7) Guarantees	✓	✓	✓	✓	✓	○[6]	✓
(R8) Responsive	○[7]	○[7]	○[7]	○[8]	✗	○[7]	✓

Requirements. We identify eight requirements that together define the Decisionhouse.

(R1) *Accessibility.* Data analysts cannot formulate integer programs; the system must provide an accessible interface [4, 25, 38, 65]. **(R2)** *Data-native.* Optimization should consume data where it lives—relational tables, Parquet files, Delta Lake—without manual extraction [14, 56]. **(R3)** *Extensibility.* The system must handle diverse optimization patterns including subset selection, resource allocation, assignment, and scheduling, with an extensible design that accommodates new constraint types. **(R4)** *Composability.* The output of one decision query must be consumable as input to another, enabling compositional workflows—echoing relational algebra's closure property. **(R5)** *Automatic planning.* The system must select algorithms based on problem structure. A supply-demand query formulated as generic MILP requires exponential-time branch-and-bound, but recognized as network flow it solves in polynomial time. **(R6)** *Governance.* Decisions must be explainable and auditable for regulatory compliance. FICO Platform [21] and Aera [1] address this in commercial settings. **(R7)** *Guarantees.* Decisions must be feasible and bounded—solvers already provide optimality gaps and feasibility certificates. **(R8)** *Responsiveness.* Optimization is NP-hard, but users expect interactive response. The system must provide progress feedback, support time-bounded execution, and return best-found solutions within user-specified budgets [2, 66].

Table 1 shows no existing system addresses all eight. OR platforms come closest by bundling solvers with data connectors and enterprise logging, but still miss automatic planning (R5) and composability (R4). Automatic planning requires detecting exploitable

gpu_pools

pool_id	capacity
H100-E	500
A100-W	300

workloads

workload_id	demand
llm-serve	200
embed-batch	250
fine-tune	150

assignments

pool_id	workload_id	cost	latency_ms
H100-E	llm-serve	10	45
H100-E	embed-batch	15	120
A100-W	embed-batch	8	85
A100-W	fine-tune	14	160

```
CREATE CANDIDATES gpu_assignments
DECISION KEY (pool_id, workload_id) AS
  SELECT a.pool_id, a.workload_id,
         p.capacity, w.demand, a.cost, a.latency_ms
  FROM assignments a
  JOIN gpu_pools p ON a.pool_id = p.pool_id
  JOIN workloads w ON a.workload_id = w.workload_id;

DECIDE allocation_plan
FROM gpu_assignments
DECISION COLUMNS (
    active   SELECTION BINARY,
    quantity CONTINUOUS BETWEEN 0 AND capacity)
WHERE latency_ms <= 200
SUBJECT TO
  CONSTRAINT pool_capacity: SUM(quantity) BY pool_id <= capacity,
  CONSTRAINT meet_demand:   SUM(quantity) BY workload_id = demand
MINIMIZE SUM(cost * quantity);
```

Figure 2: GPU allocation example: input tables (top) and DeQL workflow (bottom).

structure from the declarative formulation—solvers tune internal algorithms but cannot recognize that a MILP encodes network flow, and LLMs may suggest structure without correctness guarantees. No existing system addresses R5; it is the defining capability of the Decisionhouse. Governance (R6) is partially addressed by OR platforms through generic logging, but no system provides decision-specific audit trails.

A separate layer. Rather than extending an existing database—through UDFs as in SolveDB [56], or through engine-level operators—the Decisionhouse requires its own optimizer and execution model. (a) A SQL optimizer chooses among polynomial-time plans; the decision planner chooses among formulations that differ in *complexity class* (MILP vs. LP vs. network flow [5]). Neither UDFs nor database operators expose this choice. (b) A single `CREATE CANDIDATES` may join data from a lakehouse, forecasts from a Postgres ML pipeline, and inventory from an operational database—the Decisionhouse must federate across sources.

3 DeQL: THE DECISION QUERY LANGUAGE

SQL's `SELECT` answers "what data exists?" DeQL introduces `DECIDE` to answer "what actions to take?"—prescriptive analytics. While `PREDICT` invokes a pre-trained model, `DECIDE` triggers a translation and optimization pipeline—prescriptive queries are computationally harder and require fundamentally different infrastructure. DeQL preserves the relational model end-to-end: queries consume relations and produce relations, enabling composition with SQL and with other decision queries (R4). This section outlines the major features; the full language specification, including formal grammar, type system, and semantics, is in [13].

Running example: GPU allocation. Consider an AI inference platform allocating continuous quantities of GPU-hours from compute pools to workloads. Figure 2 shows the input tables and query: pools with available capacity, workloads with compute demand, and assignments with costs and latencies.

`CREATE CANDIDATES` (Figure 2) factors the candidate definition into a first-class database construct. `CREATE CANDIDATES` defines a named query, like a view, but annotates it with a `DECISION KEY` that declares which columns identify each *decision entity*—metadata that neither tables nor views carry. The system manages the physical representation: depending on query complexity, reuse frequency, and solver access patterns, the candidate set may remain virtual, be materialized in memory, or be persisted—analogous to how Snowflake Dynamic Tables [57] automatically manage materialization and refresh. The `DECISION KEY` resolves a structural ambiguity: the `FROM` clause produces a flat join result, but should the system create one decision variable per GPU pool, per assignment, or per row? `DECISION KEY` specifies that each unique `(pool_id, workload_id)` pair is one decision entity.

The `DECIDE` query references the named candidate set instead of repeating the join. `DECISION COLUMNS` declares what the system decides about each decision entity. Each column defines one *decision variable* per candidate: unlike data columns, whose values are read from the database, decision variables are unknowns whose values are computed at query time. They otherwise behave like regular columns—participating in expressions and appearing in the output. Each variable declares a domain: `BINARY` restricts values to 0 or 1 (select or not), while `CONTINUOUS BETWEEN 0 AND capacity` allows real values within data-driven bounds. The `BY` clause in constraints creates grouped constraints: `SUM(quantity) BY pool_id <= capacity` generates one constraint per pool. The optional `SELECTION` modifier on `active` couples a binary selection variable with the other decision columns: when `active` = 0, `quantity` is forced to zero (auto-link), aggregates over data columns automatically multiply by `active` (auto-multiply), and the `BETWEEN` bounds apply only when `active` = 1 (auto-bound). Auto-multiply targets only data columns—`SUM(cost * quantity)` involves a decision variable, so auto-link handles it instead. These three rules eliminate boilerplate that practitioners write manually, generalizing PaQL's [14] package semantics.

The `WHERE` clause in the `DECIDE` query excludes candidates before optimization—in Figure 2, assignments whose latency exceeds the SLA threshold are removed before formulation, reducing the number of decision variables the solver receives. Filters can appear in both `CREATE CANDIDATES` and `DECIDE`: in `CREATE CANDIDATES`, they remove physically infeasible candidates (e.g., GPU pools with insufficient memory for a workload); in `DECIDE`, they apply per-query policy filters that may change between invocations. The candidate set is defined once and shared; Section A.1 shows a richer scenario with hardware compatibility filters in `CREATE CANDIDATES` and `FILTER (WHERE ...)` policy constraints.

Many real-world problems require decisions at multiple granularities—for example, assigning items to batches while simultaneously determining each batch's timing. DeQL supports JOIN of candidate tables in the `DECIDE` query: each candidate table retains its own `DECISION KEY`, and decision variables are not duplicated when joined, analogous to how SQL JOINs link rows without merging table identities. This enables expressing multi-level optimization problems within a single query. The language also extends to decision-making under uncertainty: data columns can carry distributional annotations, and constraints can specify probabilistic guarantees (Section A.1).

Since DECIDE creates a standard table (like CREATE TABLE AS), decision workflows compose naturally: the output of one DECIDE becomes input to the next, and users analyze results with SQL. This relational closure enables sequential decision-making [50] within the platform—a routing decision feeds directly into a scheduling query over the chosen routes—and is a deliberate design choice: languages without it exhibit fundamental expressibility gaps [23].

The MINIMIZE clause specifies the objective to optimize. By default, the planner may only select formulations that produce optimal solutions. When users can tolerate approximate results, quality modifiers grant the planner permission to trade quality for speed: MINIMIZE SUM(cost * quantity) WITHIN 5% TIMEOUT 30s. WITHIN X% guarantees the returned objective is within X% of optimal; TIMEOUT Xs requests the best feasible solution found within a time budget. To exploit these clauses, the planner needs abstraction-based decomposition [52]: partitioning the candidate space into structured groups that each form a complete subproblem, reducing problem size while controlling approximation error.

CREATE ABSTRACTION provides the data infrastructure for this class of algorithms. It partitions the candidate space into coarser groups for methods such as SketchRefine [14] and partial-state abstraction [45]:

```
CREATE ABSTRACTION assignment_clusters
ON gpu_assignments (cost, demand, capacity) USING KMEANS(k=1000);
```

This clusters millions of candidates into thousands of representatives by cost, demand, and capacity. SketchRefine first solves a reduced problem over the representatives, then refines only the promising clusters back to individual candidates, reducing the effective problem size by orders of magnitude while bounding the approximation error. Extending abstraction-based decomposition to other problem classes and designing abstractions that yield valid solutions at any interruption point—as TIMEOUT requires—are open research challenges (Section 5).

4 DECISION QUERY PROCESSING

A Decisionhouse sits between applications and the underlying data/solver infrastructure. Data flows through four stages: a DeQL query enters the **Parser**, producing **Abstract IR**—a normalized, symbolic, database-agnostic representation with no algorithm choice and no data. The **Decision Planner** analyzes the Abstract IR, consults database statistics—cardinalities, value distributions, group structure—and selects a formulation and algorithm, producing **Concrete IR**—still symbolic, but encoding the chosen mathematical formulation and symbolic data references. The **Materializer** fetches candidate data from sources and evaluates symbolic expressions against actual rows, producing **Materialized IR**—the solver-ready coefficient vectors, constraint matrices, and variable bounds that optimization requires. The **Executor** invokes the solver and logs decisions for governance, enabling replay and what-if analysis.

4.1 The Three-IR Design

SQL systems translate queries through two intermediate representations—logical plans and physical plans [43, 49]. Decision queries need three: a symbolic query representation (Abstract IR), a formulation-specific but still symbolic representation (Concrete IR), and a data-populated instance (Materialized IR). Two properties of optimization drive the additional splits. The Abstract-to-Concrete boundary separates *what* to optimize from *how* to formulate it. SQL's logical-to-physical step selects among polynomial-time alternatives (hash join vs. merge join); the corresponding step for decision queries can change the problem's *complexity class* (MILP vs. network flow). The Concrete-to-Materialized boundary separates the symbolic formulation from the data it operates on. Different formulations require different materialization strategies: a network flow needs supply and demand vectors indexed by node; a generic MILP needs a flat coefficient matrix. Decomposition methods like SketchRefine materialize the same Concrete IR template multiple times with different data subsets. Deferring data binding until planning decisions are finalized lets the planner choose both the formulation and its materialization strategy, following established database design principles [43, 49].

Abstract IR is the normalized canonical form produced by the parser. It preserves column references, symbolic expressions (SUM(cost * quantity)), constraint structure with named constraints and grouping (BY), and decision variable declarations with types and bounds. Abstract IR encodes no algorithm choice and binds no data.

Concrete IR encodes the chosen mathematical formulation and algorithm. Different formulations produce structurally different Concrete IRs from the same Abstract IR: a generic MILP with a general constraint matrix, an LP relaxation that drops integrality constraints, or a min-cost network flow as a bipartite supply-demand graph. Solver-specific transformations further shape the IR (e.g., linearizing MAX aggregations into auxiliary variables and constraints). Concrete IR remains symbolic—it uses column references, not actual values.

Materialized IR is produced by fetching candidate data and evaluating symbolic expressions against actual rows. It provides the global data view that optimization requires—each component derived by evaluating the Concrete IR's symbolic expressions against actual rows: an objective coefficient vector $\mathbf{c}$, a constraint matrix $\mathbf{A}$ where rows correspond to constraints and columns to per-candidate decision variables, and per-variable lower- and upper-bound vectors $\mathbf{l}$ and $\mathbf{u}$. The global-view requirement constrains how decision queries scale. Exact solving demands the full dataset. Abstraction-based decomposition reduces this to structured subsets—cluster representatives or members—where each subset still requires its own complete materialization because candidates within it interact through shared constraints. SketchRefine materializes the template once for the sketch (over representatives) and once per refined cluster, enabling subproblem reuse without re-planning.

4.2 Decision Query Planning

SQL optimization selects among polynomial-time execution plans. Decision query planning can change the problem's *complexity class*: the same allocation query is NP-hard as generic MILP but polynomial as network flow (Section 4.3). We call the problem of selecting an execution plan from a declarative query the *Decision Optimization Problem* (DeOP). Solvers optimize *within* a fixed coefficient matrix; the planner optimizes *which instance* the solver receives,

using query structure and database statistics that no solver has access to.

Structure detection from Abstract IR. The Abstract IR preserves structural patterns that flat coefficient matrices obscure. In the GPU allocation query (Figure 2), the `DECISION KEY (pool_id, workload_id)` reveals bipartite structure; the `BY` constraints match supply-demand flow conservation; the objective is linear in the flow variable. Together, these identify a min-cost network flow, enabling network simplex instead of branch-and-bound (Section 4.3). Similarly, binary variables on a composite key with mutual-exclusivity constraints reveal total unimodularity [5], enabling the Hungarian algorithm [36] in $O(n^3)$. Both are deterministic pattern matches on the IR, analogous to recognizing star joins from query graphs.

Statistics-driven planning. Beyond structure detection, the planner can exploit standard database statistics. Candidate set cardinality, including join selectivity estimates for multi-table candidate spaces, determines whether to solve directly or decompose, analogous to how table size informs the choice between nested-loop and hash join. Column histograms on objective and constraint expressions characterize the coefficient landscape—information available to the database but invisible to solvers, which receive only flat coefficient matrices (Section 5).

The framework extends to stochastic settings [15, 27]: when attributes carry distributional annotations, uncertainty handling—selecting among Sample Average Approximation, chance-constraint linearization, or robust formulations—becomes another planning dimension (Section 5).

4.3 Analysis

We are building DH-0, an initial Decisionhouse implementation, in Rust, using Apache DataFusion [7] for query federation, Apache Arrow [6] as the unified columnar format across data sources and solver boundaries, and HiGHS [31] and Gurobi [26] for solving. The current release includes a custom SQL parser extended for DeQL, complete Abstract IR, Concrete IR, and Materialized IR representations, a working SketchRefine implementation, and direct solving. Data sources include DuckDB, any ODBC-compatible database, Microsoft Fabric, and all Apache DataFusion-supported backends.

Formulation selection. To evaluate automatic structure recognition, we solve the GPU allocation query from Figure 2 at seven scales up to 7.35M candidate assignments—consistent with hyperscale GPU clusters that reach $O(10^5)$ GPUs [37, 59]—using three formulations that the decision planner would select between: generic MILP (no recognition), LP relaxation (TU detection drops binary variables), and min-cost network flow (full network structure detection). Each formulation runs on HiGHS [31] with a 10-minute time limit on random instances (median of 3 runs; Figure 3).

The speedup grows superlinearly: 29× at 18K assignments, 90× at 460K, and 355× at 1.8M—from over 3 minutes to under 1 second. The speedup comes entirely from structure recognition, not approximation—all three formulations return provably optimal solutions with identical objective values at every scale where all complete. The transportation constraint matrix is totally unimodular [5], so LP relaxation produces integer solutions automatically, making binary variables redundant. Recognizing bipartite supply-demand structure further enables network simplex, changing the

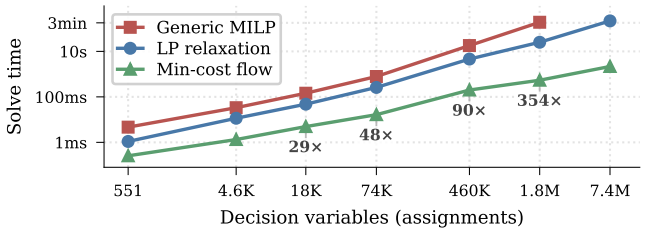


Figure 3: Solve time for the GPU allocation query (Figure 2). All three formulations produce identical optima up to 1.8M assignments; min-cost flow is 29–355× faster than MILP. At 7.4M, MILP times out (10 min); min-cost flow solves in 2.2 s.

effective complexity class. At 7.4M assignments, MILP fails to produce any feasible solution within 10 minutes; min-cost flow solves in 2.2 seconds. At scale, structure recognition determines whether the query succeeds at all—and a declarative system can detect and exploit structure the user did not specify. This experiment isolates one dimension of the DeOP: formulation selection via structure detection. Statistics-driven planning—candidate pruning, decomposition strategy, execution parameters—is a second dimension that Section 5 maps as open research.

Incremental materialization. When successive `DECIDE` queries share the same Abstract and Concrete IR—the common case in what-if analysis, where an analyst varies constraint parameters while the formulation stays fixed—only the Materialized IR changes, and only partially: bound vectors update but the constraint matrix sparsity pattern is unchanged. The system retains both the Materialized IR and the solver's internal state (preprocessed model and factored simplex basis), propagating only the changed bounds through the solver's incremental update API. The solver re-solves from its previous basis without reconstructing the model. On the GPU allocation benchmark, varying pool capacities across 50 what-if queries yields 8–15× per-query speedup in steady state, with benefit growing at larger scale (460K assignments: 8–11×; 1.8M: 12–15×).

5 RESEARCH AGENDA

Beyond the challenges raised throughout Sections 3 and 4, achieving the requirements in Section 2 raises open research questions. Additional directions include standardized benchmarks for optimization workloads, decision governance, and LLM-based query generation; we focus here on four that are tightly coupled to the Decisionhouse architecture.

Automatic reformulation and structure detection. Automatic *structure detection*—recognizing that a MILP encodes network flow, or that an assignment problem has total unimodularity—remains limited. Commercial solvers detect some structures (network rows in CPLEX), but cannot recognize semantic patterns lost in numeric encoding. The Abstract IR preserves these patterns, connecting query rewriting (predicate pushdown, partition-wise execution [54]) with OR reformulation and presolve theory [16, 53, 55].

Cost models for decision query optimization. DeOP requires predicting execution cost and solution quality before committing to a plan. MILP solve time resists closed-form characterization; prediction errors of 10–100× are common even with ML models

trained on millions of instances [33], partly because predictions operate on flat coefficient matrices with no semantic information. The three-IR design exposes a richer feature space: problem type (assignment vs. scheduling), constraint topology (bipartite keys, grouped supply-demand), candidate set size per dimension, and abstraction depth—analogous to the cardinality estimates and index statistics that make database cost models work.

Physical design for decision queries. Candidate sets introduce physical design questions absent from traditional query processing. Whether a candidate set should be virtual, in-memory, or persisted depends on join complexity, reuse across queries, solver access patterns, and data freshness. Physical layout must align with how solvers access data, not just how queries scan it: the join structure in `CREATE CANDIDATES` determines how candidates group by dimension key, defining sparsity patterns of constraint matrix blocks that can be precomputed as reusable skeletons for future `DECIDE` queries. When candidate sets derive from large joins, full materialization may be infeasible; factorized representations [48] avoid materializing joins for aggregation, but extending this to optimization—computing solver coefficients directly from factorized data—remains open.

Responsive execution. SQL scales through partitioning and streaming because tuples are processed independently; ML scales through batching because training examples contribute independently to the gradient. Optimization decisions are coupled through shared constraints, so these scaling strategies do not apply directly. Decomposition methods like SketchRefine [14] partition the candidate space into manageable subproblems, but unlike ML batching, each partition must account for coupling at its boundaries through a global coordination phase. The three-IR pipeline enables progressive refinement: the Abstract IR separates structure from data, so the planner can compute bounds at coarse abstraction levels before committing to full materialization. The pipeline also enables adaptive replanning: if the solver stalls at the Materialized IR stage, the system can fall back to a different Concrete IR without re-parsing the query, analogous to adaptive query processing [10] but across formulations rather than access paths. Extending abstraction-based decomposition beyond package queries to scheduling, network flow, and assignment problems, and designing abstractions that yield valid solutions at any interruption point, remain open.

6 RELATED WORK

Optimization tools and platforms. Commercial (Gurobi [26], CPLEX [34]) and open-source (HiGHS [31], SCIP [11]) solvers provide the algorithmic foundation. Modeling languages (AMPL [22], Pyomo [28], JuMP [20], MiniZinc [46], OR-Tools) add expressivity but require optimization expertise and lack database integration. OR platforms (FICO Xpress [21], AIMMS [12], IBM CPLEX Studio [34]) bundle these with data connectors and enterprise logging, but remain *tools for experts*: analysts must still write models in platform-specific languages, map data imperatively, and select formulations manually. Even lakehouse integration remains manual notebooks [18]. Algorithm selectors such as AutoFolio [39] choose among solvers based on instance features, but operate on already-serialized coefficient matrices; they cannot detect that a MILP encodes network flow or transportation structure. Supply chain platforms [35, 47] embed domain-specific heuristics for planning but are not general-purpose. A Decisionhouse sits above all of these, using solvers as backends while providing data-native interfaces and automatic planning—the same shift from expert tooling to declarative infrastructure that SQL databases brought to data processing.

LLMs and optimization. LLMs that translate natural-language descriptions into solver code [3, 4, 25, 29, 38, 65] provide what we call *operational intelligence*—understanding which situation to model. However, they do not provide *mathematical intelligence*: on OptiBench [63], GPT-4o formulates linear programs correctly 81% of the time but achieves below 5% accuracy on combinatorial problems requiring structure recognition. Generated formulations carry no feasibility guarantees [30, 44] and accuracy degrades with problem size. We envision LLMs as complementary front-ends—translating user intent into DeQL—while a Decisionhouse provides verified translation, formulation selection, and guaranteed solving.

Decision support systems. Classical DSS [58] provide dashboards, what-if analysis, and scenario comparison to aid human judgment. A Decisionhouse differs in kind: rather than presenting alternatives for manual evaluation, it computes optimal decisions from declarative specifications.

In-database optimization. SolveDB [56] embedded solvers in PostgreSQL; PaQL [14] pioneered declarative package queries with abstraction-based decomposition for scalability [52]—SketchRefine for millions of candidates, extended to billions by Progressive Shading [40]; PackDB [32] embeds package queries in DuckDB. These systems handle one problem class: subset selection with linear constraints over a single table. DeQL generalizes beyond packages with typed decision variables, multi-table candidate spaces, automatic formulation selection, persistent abstractions, and a three-IR pipeline that preserves problem structure through planning. Pratten et al. [51] develop algebraic foundations for subset selection and optimization within relational algebra.

7 CONCLUSION

SQL and query optimizers eliminated bespoke data-retrieval programs; Lakehouses added first-class support for ML. Deciding—computing optimal actions subject to constraints—is next. As autonomous systems take on consequential decisions over data, ad-hoc pipelines that are difficult to audit, reproduce, or govern are not sufficient. Decisionhouses provide the missing layer: every decision expressed in DeQL is declarative, constraint-enforced, and mathematically grounded—a verifiable foundation for safe and robust automated decision-making at scale.

REFERENCES

[1] Aera Technology. 2024. Aera Decision Cloud: Cognitive Automation for the Enterprise. https://www.aeratechnology.com/

[2] Sameer Agarwal, Barzan Mozafari, Aurojit Panda, Henry Milner, Samuel Madden, and Ion Stoica. 2013. BlinkDB: Queries with Bounded Errors and Bounded Response Times on Very Large Data. In *EuroSys*. 29–42.

[3] Ali AhmadiTeshnizi, Wenzhi Gao, Herman Brunborg, Shayan Talaei, Connor Lawless, and Madeleine Udell. 2024. OptiMUS-0.3: Using Large Language Models to Model and Solve Optimization Problems at Scale. *arXiv preprint arXiv:2407.19633* (2024).

[4] Ali AhmadiTeshnizi, Wenzhi Gao, and Madeleine Udell. 2024. OptiMUS: Scalable Optimization Modeling with (MI)LP Solvers and Large Language Models. In *International Conference on Machine Learning (ICML)*. 577–596.

[5] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. 1993. *Network Flows: Theory, Algorithms, and Applications.* Prentice Hall.

[6] Apache Software Foundation. 2024. Apache Arrow: A cross-language development platform for in-memory analytics. https://arrow.apache.org

[7] Apache Software Foundation. 2024. Apache DataFusion: Extensible Query Engine. https://datafusion.apache.org

[8] Michael Armbrust, Tathagata Das, Sameer Paranjpye, Reynold Xin, Shixiong Zhu, Ali Ghodsi, Burak Yavuz, Mukul Murthy, Joseph Torres, Liwen Sun, Peter Boncz, Mostafa Mokhtar, Herman Van Hovell, Adrian Ionescu, Alicja Luszczak, Michal Switakowski, Takuya Ueshin, Xiao Li, Michal Szafranski, Pieter Senster, and Matei Zaharia. 2020. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3411–3424.

[9] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1383–1394. https://doi.org/10.1145/2723372.2742797

[10] Ron Avnur and Joseph M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *SIGMOD*. 261–272.

[11] Ksenia Bestuzheva, Mathieu Besançon, Wei-Kun Chen, Antonia Chmiela, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Oliver Gaul, Gerald Gamrath, Ambros Gleixner, et al. 2021. The SCIP Optimization Suite 8.0. *ZIB-Report* (2021).

[12] Johannes Bisschop. 2006. *AIMMS – Optimization Modeling.* Lulu.com.

[13] Matteo Brucato et al. 2026. DeQL: A Decision Query Language for Prescriptive Analytics over Relational Data — Language Specification. Technical report, available at https://arxiv.org/abs/TODO.

[14] Matteo Brucato, Azza Abouzied, and Alexandra Meliou. 2018. Package Queries: Efficient and Scalable Computation of High-Order Constraints. *The VLDB Journal* 27, 5 (2018), 693–718.

[15] Matteo Brucato, Nishant Yadav, Azza Abouzied, Peter J. Haas, and Alexandra Meliou. 2020. Stochastic Package Queries in Probabilistic Databases. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020*. ACM, 269–283. https://doi.org/10.1145/3318464.3389765

[16] Michele Conforti, Gérard Cornuéjols, and Giacomo Zambelli. 2010. Extended Formulations in Combinatorial Optimization. *4OR* 8, 1 (2010), 1–48.

[17] Benoît Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 215–226.

[18] Databricks. 2024. Assembling Toy Brick Sets with Gurobi & Databricks: A Gentle Introduction to Optimization. https://www.databricks.com/blog/assembling-toy-brick-sets-gurobi-databricks-gentle-introduction-optimization

[19] Databricks. 2024. MLflow: An Open Source Platform for the Machine Learning Lifecycle. https://mlflow.org

[20] Iain Dunning, Joey Huchette, and Miles Lubin. 2017. JuMP: A Modeling Language for Mathematical Optimization. *SIAM Rev.* 59, 2 (2017), 295–320.

[21] FICO. 2024. FICO Platform: Enterprise Decision Management. https://www.fico.com/en/fico-platform

[22] Robert Fourer, David M. Gay, and Brian W. Kernighan. 2002. *AMPL: A Modeling Language for Mathematical Programming* (2nd ed.). Cengage Learning.

[23] Amélie Gheerbrant, Leonid Libkin, Liat Peterfreund, and Alexandra Rogova. 2025. GQL and SQL/PGQ: Theoretical Models and Expressive Power. *Proc. VLDB Endow.* 18, 6 (2025), 1798–1810. https://doi.org/10.14778/3725688.3725707

[24] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. 2011. Dominant Resource Fairness: Fair Allocation of Multiple Resource Types. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2011, Boston, MA, USA, March 30 - April 1, 2011*. USENIX Association. https://www.usenix.org/conference/nsdi11/dominant-resource-fairness-fair-allocation-multiple-resource-types

[25] Gurobi Optimization. 2024. Gurobot. https://www.gurobi.com/solutions/gurobot/

[26] Gurobi Optimization, LLC. 2024. Gurobi Optimizer Reference Manual. https://www.gurobi.com

[27] Riddho R. Haque, Anh L. Mai, Matteo Brucato, Azza Abouzied, Peter J. Haas, and Alexandra Meliou. 2025. Stochastic SketchRefine: Scaling In-Database Decision-Making under Uncertainty to Millions of Tuples. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3106–3118.

[28] William E. Hart, Jean-Paul Watson, and David L. Woodruff. 2011. Pyomo: modeling and solving mathematical programs in Python. *Mathematical Programming Computation* 3, 3 (2011), 219–260.

[29] Chenyu Huang, Zhengyang Tang, Shixi Hu, Ruoqing Jiang, Xin Zheng, Dongdong Ge, Benyou Wang, and Zizhuo Wang. 2025. ORLM: A Customizable Framework in Training Large Models for Automated Optimization Modeling. *Oper. Res.* 73, 6 (2025), 2986–3009. https://doi.org/10.1287/OPRE.2024.1233

[30] Sen Huang, Kaixiang Yang, Sheng Qi, and Rui Wang. 2024. When Large Language Model Meets Optimization. *arXiv preprint arXiv:2405.10098* (2024).

[31] Qi Huangfu and J. A. Julian Hall. 2018. Parallelizing the dual revised simplex method. *Mathematical Programming Computation* 10, 1 (2018), 119–142.

[32] HuDa Lab. 2025. PackDB: In-Database Package Query Optimization over DuckDB. https://huda-lab.github.io/packdb. Software, MIT License.

[33] Frank Hutter, Lin Xu, Holger H. Hoos, and Kevin Leyton-Brown. 2014. Algorithm Runtime Prediction: Methods & Evaluation. *Artificial Intelligence* 206 (2014), 79–111.

[34] IBM. 2024. IBM ILOG CPLEX Optimization Studio. https://www.ibm.com/products/ilog-cplex-optimization-studio

[35] Kinaxis. 2024. Introducing Maestro: The First AI-Infused Supply Chain Orchestration Platform. https://www.kinaxis.com/en/resources/blog/introducing-maestro

[36] Harold W. Kuhn. 1955. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly* 2, 1–2 (1955), 83–97.

[37] Neeraj Kumar, Pol Mauri Ruiz, Vijay Menon, Igor Kabiljo, Mayank Pundir, Andrew Newell, Daniel Lee, Liyuan Wang, and Chunqiang Tang. 2024. Optimizing Resource Allocation in Hyperscale Datacenters: Scalability, Usability, and Experiences. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024*. USENIX Association, 507–528.

[38] Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. 2023. Large Language Models for Supply Chain Optimization. *arXiv preprint arXiv:2307.03875* (2023).

[39] Marius Lindauer, Holger H. Hoos, Frank Hutter, and Torsten Schaub. 2015. AutoFolio: An Automatically Configured Algorithm Selector. *Journal of Artificial Intelligence Research* 53 (2015), 745–778.

[40] Anh L. Mai, Pengyu Wang, Azza Abouzied, Matteo Brucato, Peter J. Haas, and Alexandra Meliou. 2024. Scaling Package Queries to a Billion Tuples via Hierarchical Partitioning and Customized Optimization. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1146–1158.

[41] Alexandra Meliou, Azza Abouzied, Peter J. Haas, Riddho R. Haque, Anh Mai, and Vasileios Vittis. 2025. Data Management Perspectives on Prescriptive Analytics. In *International Conference on Database Theory (ICDT)*. 2:1–2:12. Invited Talk.

[42] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. 2010. Dremel: Interactive Analysis of Web-Scale Datasets. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 330–339.

[43] Hubert Mohr-Daurat, Xuan Sun, and Holger Pirk. 2023. BOSS - An Architecture for Database Kernel Composition. *Proc. VLDB Endow.* 17, 4 (2023), 877–890. https://doi.org/10.14778/3636218.3636239

[44] Mahdi Mostajabdaveh, Timothy Tin Long Yu, Samarendra Chandan Bindu Dash, Rindra Ramamonjison, Jabo Serge Byusa, Giuseppe Carenini, Zirui Zhou, and Yong Zhang. 2025. Evaluating LLM Reasoning in the Operations Research Domain with ORQA. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*. AAAI Press, 24902–24910. https://doi.org/10.1609/AAAI.V39I23.34673

[45] Samer B. Nashed, Justin Svegliato, Matteo Brucato, Connor Basich, Rod Grupen, and Shlomo Zilberstein. 2021. Solving Markov Decision Processes with Partial State Abstractions. In *IEEE International Conference on Robotics and Automation (ICRA)*. 813–819.

[46] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. 2007. MiniZinc: Towards a Standard CP Modelling Language. In *Principles and Practice of Constraint Programming – CP 2007*. Springer, 529–543.

[47] o9 Solutions. 2024. AI-Powered Integrated Business Planning Platform. https://o9solutions.com/

[48] Dan Olteanu and Maximilian Schleich. 2016. Factorized Databases. *SIGMOD Rec.* 45, 2 (2016), 5–16. https://doi.org/10.1145/3003665.3003667

[49] Pedro Pedreira, Orri Erling, Konstantinos Karanasos, Scott Schneider, Wes McKinney, Satyanarayana R. Valluri, Mohamed Zaït, and Jacques Nadeau. 2023. The Composable Data Management System Manifesto. *Proc. VLDB Endow.* 16, 10 (2023), 2679–2685. https://doi.org/10.14778/3603581.3603604

[50] Warren B. Powell. 2022. *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions.* John Wiley & Sons, Hoboken, NJ.

[51] David Robert Pratten, Luke Mathieson, and Fahimeh Ramezani. 2025. Relational Algebras for Subset Selection and Optimisation. *arXiv preprint arXiv:2509.06439* (2025).

[52] David F. Rogers, Robert D. Plante, Richard T. Wong, and James R. Evans. 1991. Aggregation and Disaggregation Techniques and Methodology in Optimization. *Operations Research* 39, 4 (1991), 553–582.

[53] Martin W. P. Savelsbergh. 1994. Preprocessing and Probing Techniques for Mixed Integer Programming Problems. *ORSA Journal on Computing* 6, 4 (1994), 445–454.

[54] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *SIGMOD*. 23–34. https://doi.org/10.1145/582095.582099

[55] Hanif D. Sherali and Warren P. Adams. 1999. *A Reformulation-Linearization Technique for Solving Discrete and Continuous Nonconvex Problems.* Springer.

[56] Laurynas Siksnys and Torben Bach Pedersen. 2016. SolveDB: Integrating Optimization Problem Solvers Into SQL Databases. In *Proceedings of the 28th International Conference on Scientific and Statistical Database Management*. ACM, 14:1–14:12.
[57] Daniel Sotolongo, Daniel Mills, Tyler Akidau, Anirudh Santhiar, Attila-Péter Tóth, Botong Huang, Boyuan Zhang, Igor Belianski, Ling Geng, Matt Uhlar, Nikhil Shah, Olivia Zhou, Saras Nowak, Sasha Lionheart, Vlad Lifliand, Wendy Grus, Yiwen Zhu, Ankur Sharma, Dzmitry Pauliukevich, Enrico Sartorello, Ilaria Battiston, Ivan Kalev, Lawrence Benson, Leon Papke, Niklas Semmler, Till Merker, and Yi Huang. 2025. Streaming Democratized: Ease Across the Latency Spectrum with Delayed View Semantics and Snowflake Dynamic Tables. In *SIGMOD Conference Companion*. ACM, 622–634. https://doi.org/10.1145/3722212.3724455
[58] Ralph H. Sprague, Jr. 1980. A Framework for the Development of Decision Support Systems. *MIS Q.* 4, 4 (1980), 1–26. https://doi.org/10.2307/248957
[59] Chunqiang Tang. 2025. Meta's Hyperscale Infrastructure: Overview and Insights. *Commun. ACM* 68, 2 (2025), 52–63. https://doi.org/10.1145/3701296
[60] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems, EuroSys 2015, Bordeaux, France, April 21-24, 2015*. ACM, 18:1–18:17. https://doi.org/10.1145/2741948.2741964
[61] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022*. USENIX Association, 945–960. https://www.usenix.org/conference/nsdi22/presentation/weng
[62] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. 2020. AntMan: Dynamic Scaling on GPU Clusters for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 533–548. https://www.usenix.org/conference/osdi20/presentation/xiao
[63] Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Wei Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. 2025. OptiBench Meets ReSocratic: Measure and Improve LLMs for Optimization Modeling. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. https://openreview.net/forum?id=fsDZwS49uY
[64] Matei Zaharia, Ali Ghodsi, Reynold Xin, and Michael Armbrust. 2021. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *CIDR*.
[65] Xinzhi Zhang, Zeyi Chen, Humishka Zope, Hugo Barbalho, Konstantina Mellou, Marco Molinaro, Janardhan Kulkarni, Ishai Menache, and Sirui Li. 2025. OptiMind: Teaching LLMs to Think Like Optimization Experts. *arXiv preprint arXiv:2509.22979* (2025).
[66] Shlomo Zilberstein. 1996. Using Anytime Algorithms in Intelligent Systems. *AI Magazine* 17, 3 (1996), 73–83.

A DEQL LANGUAGE DETAILS

This appendix provides additional DeQL syntax examples and demonstrates backward compatibility with PaQL.

A.1 Extended Syntax Examples

This section provides syntax examples for common optimization patterns expressible in DeQL. Subset selection (the simplest pattern) is covered in Section A.2.

Continuous Allocation with SELECTION: Portfolio Optimization. Select 10–20 stocks from a universe of companies, with per-stock weight allocation and sector diversification caps (Section 4.2 walks through the planner's decisions on this query). The `SELECTION BINARY` variable `is_selected` auto-links the continuous `weight` to zero for unselected stocks; auto-bound ensures the 1–8% weight range applies only when `is_selected` = 1.

```
DECIDE portfolio
FROM companies
DECISION COLUMNS (
    is_selected SELECTION BINARY,
    weight      CONTINUOUS BETWEEN 0.01 AND 0.08
)
SUBJECT TO
  CONSTRAINT fully_invested: SUM(weight) = 1.0,
  CONSTRAINT min_holdings:   SUM(is_selected) >= 10,
  CONSTRAINT max_holdings:   SUM(is_selected) <= 20,
  CONSTRAINT cap_tech:       SUM(weight) <= 0.25
    FILTER (WHERE sector = 'Technology'),
  CONSTRAINT cap_health:     SUM(weight) <= 0.20
    FILTER (WHERE sector = 'Health Care'),
  CONSTRAINT min_income:
    SUM(dividend_yield * weight) >= 0.015
MAXIMIZE SUM((earnings_per_share / price) * weight);
```

Bipartite Matching. Match items from one set to another:

```
CREATE CANDIDATES worker_shifts
DECISION KEY (worker_id, shift_id)
AS SELECT w.worker_id, s.shift_id, w.skill_level, s.required_skill
   FROM Workers w CROSS JOIN Shifts s
   WHERE w.skill_level >= s.required_skill;

DECIDE schedule
FROM worker_shifts
SUBJECT TO
  CONSTRAINT one_shift: COUNT(*) BY worker_id <= 1,
  CONSTRAINT one_worker: COUNT(*) BY shift_id = 1
MINIMIZE SUM(ABS(skill_level - required_skill));
```

Job-Shop Scheduling. The classic job-shop problem: multiple tasks compete for shared machines. The `DISJUNCTIVE` constraint ensures tasks assigned to the same machine do not overlap.

```
CREATE CANDIDATES jobs
DECISION KEY (job_id)
DECISION COLUMNS (start_time CONTINUOUS, machine INTEGER)
AS SELECT job_id, processing_time, deadline, priority
   FROM ProductionJobs WHERE status = 'pending';

DECIDE production_plan
FROM jobs
SUBJECT TO
  CONSTRAINT deadline: start_time + processing_time <= deadline,
  CONSTRAINT no_overlap: DISJUNCTIVE(start_time, processing_time)
      BY machine
MINIMIZE MAX(start_time + processing_time);
```

Realistic GPU Allocation: Filters, Multi-Resource, and QoS. The main body (Figure 2) shows a simplified GPU allocation query for presentation. Real GPU cluster schedulers involve hardware compatibility checks, multiple resource dimensions, and QoS-tier policies. This example extends the main body schema with columns grounded in the GPU scheduling literature: `gpu_type` and `memory_gb` on pools (multi-resource scheduling [24]), `min_memory_gb` and `qos_class` on workloads (QoS tiers from AntMan [62] and Alibaba production clusters [61]), and `mem_per_unit` for per-workload memory consumption.

```
-- Hardware compatibility in CREATE CANDIDATES:
-- pools without enough GPU memory are excluded entirely.
-- These are physically infeasible pairings, analogous to
-- Borg's feasibility checking and Kubernetes node selectors.
CREATE CANDIDATES gpu_assignments
DECISION KEY (pool_id, workload_id) AS
  SELECT a.pool_id, a.workload_id,
         p.capacity, p.memory_gb,
         w.demand, w.qos_class, w.mem_per_unit,
         a.cost, a.latency_ms
  FROM assignments a
  JOIN gpu_pools p ON a.pool_id = p.pool_id
  JOIN workloads w ON a.workload_id = w.workload_id
  WHERE p.memory_gb >= w.min_memory_gb;

-- Policy-driven allocation
DECIDE allocation_plan
FROM gpu_assignments
DECISION COLUMNS (
    active   SELECTION BINARY,
    quantity CONTINUOUS BETWEEN 0 AND capacity)
WHERE latency_ms <= 200
SUBJECT TO
  CONSTRAINT pool_capacity:
    SUM(quantity) BY pool_id <= capacity,
  CONSTRAINT mem_capacity:
```

```
    SUM(mem_per_unit * quantity) BY pool_id <= memory_gb,
  CONSTRAINT meet_demand:
    SUM(quantity) BY workload_id = demand,
  CONSTRAINT sla_reserve:
    SUM(quantity) FILTER (WHERE qos_class = 'realtime')
    BY pool_id >= 0.4 * capacity
MINIMIZE SUM(cost * quantity);
```

Three levels of filtering serve distinct roles. (1) `WHERE` in `CREATE CANDIDATES` removes physically infeasible pairings—a GPU pool whose memory is below a workload's minimum requirement can never host that workload, regardless of policy. This parallels Borg's feasibility check [60] and Kubernetes node selectors: stable hardware constraints that define the feasible domain, computed once and shared across scheduling rounds. (2) `WHERE` in `DECIDE` applies a per-query policy filter. The SLA latency threshold may vary between scheduling rounds (e.g., relaxing to 300 ms during off-peak hours); unlike the hardware filter, it does not alter the reusable candidate set. (3) `FILTER (WHERE ...)` in a constraint selects which candidates participate in a specific aggregation without eliminating any candidate from the problem. The `sla_reserve` constraint requires that at least 40% of each pool's allocated capacity goes to realtime workloads—a QoS reservation policy grounded in AntMan's resource-guarantee tiers [62] and Kubernetes QoS classes. Batch workloads remain eligible for the remaining capacity. The `mem_capacity` constraint adds a second resource dimension (GPU memory alongside GPU-hours), reflecting dominant resource fairness [24] where schedulers must respect multiple resource types simultaneously.

Multi-Granularity: Batch Scheduling. Some problems require decisions at multiple granularities. Here, items must be assigned to batches (item-level decisions), while each batch has its own timing (batch-level decisions). Two candidate tables with different `DECISION KEY`s are joined in a single `DECIDE` query; decision variables are not duplicated across join rows.

```
-- Item-level: assign each item to a batch
CREATE CANDIDATES item_batch_assignments
DECISION KEY (item_id, batch_id) AS
  SELECT i.item_id, b.batch_id, i.weight, i.deadline,
         b.capacity
  FROM items i CROSS JOIN batches b;

-- Batch-level: determine timing for each batch
CREATE CANDIDATES batch_timing
DECISION KEY (batch_id) AS
  SELECT batch_id, capacity, processing_time
  FROM batches;

DECIDE schedule
FROM item_batch_assignments a
  JOIN batch_timing t ON a.batch_id = t.batch_id
DECISION COLUMNS (
    assigned  SELECTION BINARY ON a,
    start_time CONTINUOUS BETWEEN 0 AND 1440 ON t)
SUBJECT TO
  CONSTRAINT one_batch:
    SUM(assigned) BY item_id = 1,
  CONSTRAINT batch_cap:
    SUM(weight * assigned) BY batch_id <= capacity,
  CONSTRAINT item_deadline:
    start_time + processing_time <= deadline,
  CONSTRAINT sequence:
    DISJUNCTIVE(start_time, processing_time) BY batch_id
MINIMIZE MAX(start_time + processing_time);
```

The `ON` clause in `DECISION COLUMNS` specifies which candidate table each variable belongs to. The `assigned` variable is indexed by `(item_id, batch_id)`, while `start_time` is indexed by `batch_id` alone—the join does not duplicate `start_time` across item rows.

Decision-Making under Uncertainty. In the deterministic GPU allocation query (Figure 2), workload demand is a fixed value. In practice, inference demand is inherently uncertain: traffic is bursty, new model deployments shift load unpredictably, and demand forecasts are derived from historical logs that capture means and variances rather than exact future values. DeQL extends to stochastic settings with three constructs:

```
CREATE CANDIDATES gpu_assignments
DECISION KEY (pool_id, workload_id) AS
  SELECT a.pool_id, a.workload_id,
         p.capacity, a.cost, a.latency_ms,
         STOCHASTIC('normal', w.demand_mean, w.demand_std)
           AS demand
  FROM assignments a
  JOIN gpu_pools p ON a.pool_id = p.pool_id
  JOIN workloads w ON a.workload_id = w.workload_id;

DECIDE allocation_plan
FROM gpu_assignments
DECISION COLUMNS (
    active   SELECTION BINARY,
    quantity CONTINUOUS BETWEEN 0 AND capacity)
WHERE latency_ms <= 200
SUBJECT TO
  CONSTRAINT pool_capacity: SUM(quantity) BY pool_id <= capacity,
  CONSTRAINT meet_demand:   SUM(quantity) BY workload_id >= demand
    WITH PROBABILITY >= 0.95
MINIMIZE EXPECTED SUM(cost * quantity);
```

`STOCHASTIC('normal', mean, std)` declares that `demand` is drawn from a distribution parameterized by data columns—a normal distribution is a reasonable first approximation when historical demand is roughly symmetric; heavier-tailed models (log-normal for right-skewed burst traffic, or empirical distributions from historical traces) use the same syntax with a different distribution identifier. `WITH PROBABILITY >= 0.95` turns `meet_demand` into a chance constraint: satisfy each workload's demand in at least 95% of scenarios. `EXPECTED` on the objective optimizes expected cost across scenarios. The system compiles this to a Sample Average Approximation (SAA) by generating scenarios from the declared distributions; the decision planner selects among SAA, chance-constraint linearization [27], and robust formulations—another planning dimension, analogous to selecting among MILP, LP relaxation, and network flow for deterministic queries. Prior work on stochastic package queries [15, 27] demonstrates feasibility at scale.

A.2 Backward Compatibility with PaQL

Every PaQL package query has a direct DeQL equivalent. Below, a PaQL query that selects up to `max_servings` servings of gluten-free recipes is translated to DeQL:

PaQL:

```
SELECT PACKAGE(*) AS R FROM Recipes
REPEAT max_servings
WHERE R.gluten = 0
SUCH THAT COUNT(*) BETWEEN 3 AND 5
  AND SUM(kcal) >= 2000
MINIMIZE SUM(fat)
```

DeQL:

```
CREATE CANDIDATES gluten_free_recipes
DECISION KEY (recipe_id)
AS SELECT recipe_id, kcal, fat, max_servings
   FROM Recipes WHERE gluten = 0;

DECIDE meal_plan
FROM gluten_free_recipes
DECISION COLUMNS (
    selected SELECTION BINARY,
    servings INTEGER BETWEEN 1 AND max_servings)
```

```
SUBJECT TO
  CONSTRAINT min_recipes: SUM(selected) >= 3,
  CONSTRAINT max_recipes: SUM(selected) <= 5,
  CONSTRAINT calories: SUM(kcal * servings) >= 2000
MINIMIZE SUM(fat * servings);
```

PaQL's `REPEAT max_servings` implicitly creates an integer multiplicity variable per tuple and applies auto-multiply to all aggregates. DeQL makes variables explicit: `SELECTION BINARY` on `selected` provides auto-link (when `selected = 0`, `servings` is forced to 0) and auto-bound (`servings` bounds apply only when `selected = 1`)—semantics PaQL lacks. The `CREATE CANDIDATES` separation enables pre-query optimization, and named constraints (`min_recipes`, `calories`) support governance and debugging.